

El Pipeline Definitivo: De la Web al Modelo Modelo Predictivo

Entendiendo el verdadero trabajo del Data Scientist a través de APIs y Data Wrangling.

CODERHOUSE

La Ilusión del 10%



Mito:

"Los datos vienen listos para aplicar Machine Learning."

Realidad:

Los datos del mundo real vienen incompletos, anidados, desorganizados y contaminados.

El Concepto:

Obtener el dato es solo la materia prima. El **Data Wrangling** (o **Data Munging**) es la fábrica que transforma esa materia prima heterogénea en un producto confiable y analizable.

La Extracción: El Transporte de Materia Prima



La Traducción: Aplanando el Caos

The Chaos

{ [{ }] } { }

{ [] { [] } } [] }

{ } } [{ }] { }

Python requests

pd.DataFrame()

The Order

	Column A	Column B	Column C
Row 1	...	...	...
Row 2	...	...	...
Row 3	...	...	...
Row 4	...	...	...
Row 5	...	...	...

El Problema:

JSON habla en árboles y ramas (datos anidados).
Machine Learning habla en matrices (filas y columnas).

El Puente:

Usamos la librería requests para hacer el GET, evaluamos el Status Code (200 = OK), extraemos el .json() y lo inyectamos directamente en un pd.DataFrame().

El Resultado:

Nuestra materia prima ya está en la línea de ensamblaje tabular.

La Línea de Ensamblaje: 6 Etapas del Data Wrangling



1. Descubrimiento

Conocer antes de tocar. ¿De dónde vienen? ¿Hay nulos? (`df.info()`).

2. Estructuración

Tipos coherentes (ej. texto a datetime). Wide vs. Long reshaping.

3. Limpieza

El filtro crítico. Imputar/eliminar nulos, deduplicar, normalizar texto, tratar outliers.

4. Enriquecimiento

Agregar valor. Cruzar variables con datos demográficos externos.

5. Validación

Control de calidad. Comprobar que las transformaciones no introdujeron errores.

6. Publicación

Empaquetar el dataset limpio (CSV, BD) listo para análisis o IA.

Integración en la Fábrica: `merge()` vs. `concat()`

`merge()` (El Rompecabezas)



Funciona como un JOIN de SQL. Combina distintos DataFrames horizontalmente utilizando una columna o clave en común (Ej: unir Ventas con Clientes usando `id_cliente`).

`concat()` (El Tetris)



Sirve para apilar DataFrames (generalmente por filas). Ideal para unir datos con la misma estructura (Ej: apilar ventas de Enero.csv sobre Febrero.csv).

Diagnóstico: ¿Wrangling o Preprocesamiento?

	Data Wrangling (Preparación General)	Machine Learning Preprocessing (Adaptación Algorítmica)
Objetivo:	Hacer el dato comprensible para el humano y el negocio.	Hacer el dato digerible para el algoritmo matemático.
Acciones:	Corregir errores, tratar nulos, cruzar tablas, normalizar textos, estructurar fechas.	Escalado de variables (MinMax, StandardScaler), Encoding de categóricas (One-Hot, Label Encoding).
Resultado:	Una tabla limpia, confiable e interpretable.	Una matriz puramente numérica (Tensores/Arrays) sin contexto de negocio directo.

Todo modelo necesita Wrangling previo, pero no todo Wrangling termina en Machine Learning.

Coder Tips

El Framework del Científico de Datos Profesional

1. Conoce el Dominio del Negocio

No decidas solo desde el código. ¿Un '0' en ventas es un valor nulo, un error del sistema, o un día que la tienda cerró? El contexto manda.

2. Diccionario de Datos

Nunca asumas qué es una variable. Documenta: Nombre, Tipo, y Descripción exacta para evitar que diferentes áreas interpreten la misma métrica de forma distinta.

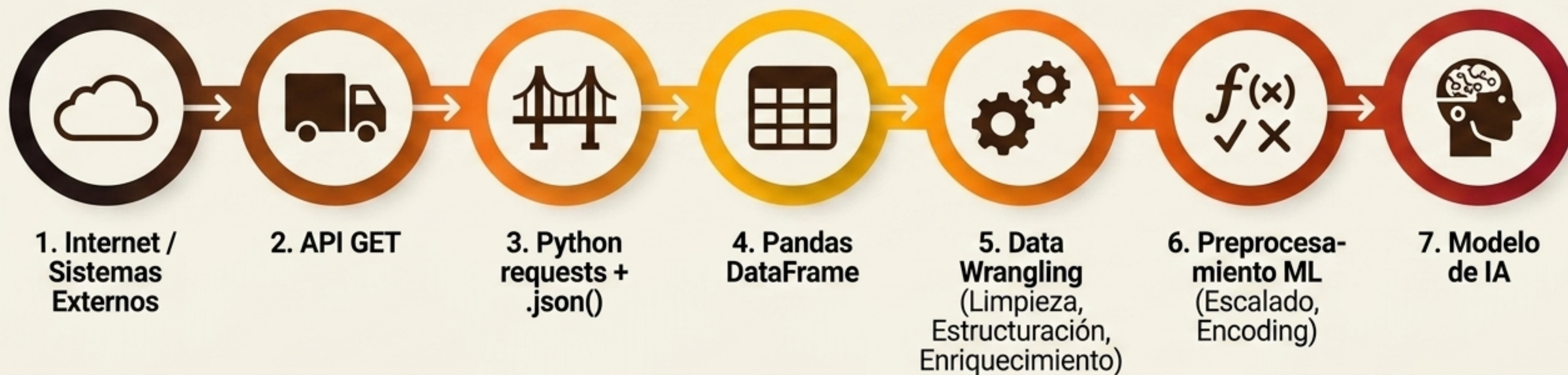
3. Trazabilidad del Origen

Antes de limpiar, conoce: ¿Dónde está alojado? ¿Cuál es el sistema fuente? ¿Cuál es su formato original?

4. Control de Versiones

Mantén copias. Nunca destruyas ni sobrescribas el dataset crudo original. Si la transformación falla, debes poder volver atrás.

El Pipeline Completo: El Viaje del Dato



El código es solo la herramienta; la verdadera habilidad es
orquestrar este flujo con criterio lógico y de negocio.

Momentos de la Clase:

#FindTheBug

Analiza este código erróneo
para hallar el defecto en
nuestro pipeline.

```
import requests
import pandas as pd

url = 'https://api.empresa.com/v1/ventas'
headers = {'Authorization': 'Bearer MI_API_KEY'}

# Haciendo la llamada a la API
respuesta = requests.get(url, headers=headers)

# Procesando los datos crudos
datos_json = respuesta.json()
df = pd.DataFrame(datos_json['resultados'])
df.head()
```

¿Qué paso crítico y profesional nos saltamos antes de intentar convertir la respuesta en JSON?
(Hint: ¿Qué pasa si excedimos el Rate Limit o el servidor cayó?)